Plan de pruebas — RutaKids (arquitectura híbrida)

1. Introducción

RutaKids es un juego móvil 2D para niñas y niños de 4 a 13 años, hecho para Fundación ADO. Enseña los puntos del decálogo de movilidad mientras el niño maneja un auto por una ciudad basada en Puebla.

El equipo está evaluando una **arquitectura híbrida**:

- **App nativa** en cada plataforma: Swift/SwiftUI en iOS y Kotlin/Jetpack Compose en Android. Se encarga del menú, la selección de niveles, la selección de edad, la pantalla de tutores, el modo kiosco y el guardado del progreso.
- **Godot 4.7.2 como módulo incrustado:** corre solo los niveles (manejo, reglas, copiloto y tarjeta de hábito). En iOS se incrusta con SwiftGodotKit y en Android con `godot-lib` 4.7.2.stable y un plugin v2.
- **Puente `RutaBridge`:** comunica los dos lados con mensajes JSON versionados (`"v": 1`). El nativo emite 5 señales hacia Godot (`start_level`, `pause_level`, `resume_level`, `abort_level`, `set_muted`) y Godot responde con un solo método, `notify(event, json)`, usando los eventos `engine_ready`, `level_started`, `level_finished`, `level_exited`, `inactivity_timeout`, `tutor_requested` y `bridge_error`.

---

2. Objetivos

1. **Validar el puente `RutaBridge` en ambas plataformas:** que los mensajes viajen de ida y vuelta, con los mismos nombres y el mismo formato JSON en iOS y Android (hallazgo H5).
2. **Verificar el ciclo de vida completo de un nivel:** menú nativo → `start_level` → `level_started` → `level_finished` (el nativo guarda sin cerrar la vista) → tarjeta de hábito → `level_exited` → regreso a `idle.tscn` y al menú con las estrellas actualizadas.
3. **Comprobar la estabilidad de Godot incrustado:** se inicia una sola vez por sesión y nunca se destruye entre niveles (H1). Abrir y cerrar un nivel 10 veces seguidas, sin cierres y sin que la memoria siga subiendo (H7).
4. **Medir los requisitos no funcionales en celulares reales de gama baja:**
   - El menú nativo aparece en menos de 3 segundos (HU-01).
   - Tiempo hasta `engine_ready`.
   - Tamaño de la app en build *release* (H4, HU-03).
   - Memoria usada durante el juego.

5. **Verificar el manejo de errores del contrato:** JSON inválido, versión no soportada, nivel inexistente (`unknown_level`), nivel sin escena (`level_not_built`), campos inválidos, y cierre de un nivel mientras todavía carga (llega `level_exited` sin `level_started`).
6. **Verificar el comportamiento ante interrupciones:** app en segundo plano o llamada entrante (`pause_level` / `resume_level`), sin perder la pausa de una falta ni la de la tarjeta de hábito.
7. **Confirmar que se cumplen las reglas que nunca se rompen:** sin conexión a internet (REQ-5), sin datos personales (REQ-6) y guardado solo de estrellas, puntos, faltas por regla y ajustes.
8. **Asegurar que las reglas del juego den el mismo resultado en las dos plataformas:** mismas faltas, puntos y estrellas para la misma forma de manejar.

---

3. Alcance

3.1 Elementos que se prueban

Componente	iOS + Godot	Android + Godot
App nativa (menú, selección de niveles, guardado de estrellas)	SwiftUI + SwiftGodotKit	Compose + `godot-lib` 4.7.2
Puente `RutaBridge` (5 señales + `notify`)	Clase Swift `@Godot`	Plugin v2 en Kotlin (`RutaBridgePlugin.kt`)
Lado de Godot del puente	`autoload/bridge_client.gd` (mismo código en las dos)	igual
Niveles 0, 1, 2 y nivel de prueba 99	sí	sí
Reglas: alto, paso peatonal, semáforo, límite y zona escolar, ciclovía	sí	sí
Copiloto (Feedback), tarjeta de hábito, HUD	sí	sí
Datos de nivel (`data/level_XX.tres`)	sí	sí

3.2 Funciones y características que se prueban

- **Arranque:** Godot inicia en segundo plano con la vista oculta mientras el menú nativo ya se puede usar; el botón del nivel muestra un indicador de carga hasta `engine_ready`.
- **Flujo de nivel:** `level_finished` incluye `stars`, `points`, `infractions` por regla y `duration_s` (sin pausas). **Repetir** emite `level_started` otra vez y **Continuar** emite `level_exited` con `completed: true`.
- **Interrupciones:** segundo plano, `abort_level`, un `start_level` encima de un nivel abierto, `set_muted` y kiosco (`inactivity_timeout` una sola vez).
- **Errores:** todos los códigos de `bridge_error`.
- **Rendimiento:** tiempos, memoria y tamaño en dispositivos reales.
- **Paridad:** mismos resultados del puente en iOS y en Android.

3.3 Tipos de prueba

#	Tipo de prueba	Qué cubre	Técnica / herramienta	Esta
1	Validación de requerimientos	Un caso de prueba por cada REQ y cada HU del SRS	Matriz de trazabilidad (3.3.1), `Pruebas_de_Aceptacion_RutaKids.xlsx`	En d
2	Prueba alfa (evaluación heurística)	Interfaz del menú nativo, HUD, copiloto y tarjeta de hábito	10 heurísticas de Nielsen adaptadas a niños	En d
3	Regresión	Que lo que ya funcionaba siga funcionando después de cada cambio	Re-ejecución de las pruebas automáticas y de los casos de aceptación	**Pend hast tene códi**
4	Dinámicas (caja blanca)	Rutas internas de los módulos principales	Cobertura de decisiones y caminos con escenas de prueba en Godot sin interfaz y `FakeBridge`	En d
5	Funcionales (caja negra)	Comportamiento visible de cada regla y del puente	Particiones de equivalencia y valores límite	En d
6	Usabilidad con usuarios	Niñas y niños de 4 a 7 y de 8 a 13 años, tutores y personal de ADO	Tareas observadas con métricas	**Pend hast tene códi**

#	Tipo de prueba	Qué cubre	Técnica / herramienta	Esta
7	No funcionales	Seguridad y privacidad, rendimiento, compatibilidad	Captura de tráfico, revisión de permisos y almacenamiento, mediciones en dispositivo	En d

3.3.1 Prueba de validación de requerimientos

Cada requerimiento y cada historia de usuario del SRS tiene al menos un caso de prueba. El detalle de pasos y resultados esperados va en `Pruebas_de_Aceptacion_RutaKids.xlsx`.

Caso	Requerimiento / HU	Se verifica que…
TC-REQ-01	REQ-1 Despliegue multiplataforma	La misma versión del juego corre en Android (arm64 y armv7) y en iOS
TC-REQ-02	REQ-2 Modo kiosco	Con `kiosk: true` todos los niveles abren, no se guarda progreso y llega `inactivity_timeout` a los 60 s
TC-REQ-03	REQ-3 Instalación en dispositivos personales	La app se instala y abre en un celular personal Android y en un iPhone
TC-REQ-04	REQ-4 Gama media/baja	En el Galaxy A15 el juego corre a 30 fps o más y sin cierres
TC-REQ-05	REQ-5 Sin internet	Con modo avión, todo funciona igual; no hay tráfico de red
TC-REQ-06	REQ-6 Privacidad	No se pide ni se guarda ningún dato personal; solo estrellas, puntos, faltas por regla y ajustes
TC-REQ-07	REQ-7 Minijuegos del decálogo	Los niveles 0, 1 y 2 se juegan de principio a fin y cada uno enseña su punto del decálogo
TC-REQ-08	REQ-8 Interfaz adaptada a la edad	Botones grandes, mensajes cortos con voz, luces del semáforo distinguibles por posición
TC-REQ-09	REQ-9 Acceso directo	La app abre directo en el menú, sin registro ni pantallas previas

Caso	Requerimiento / HU	Se verifica que…
TC-REQ-10	REQ-10 Retroalimentación pedagógica	Cada falta muestra al copiloto con el porqué; cada acierto da sonido y "+10"
TC-HU-01	HU-01 (REQ-6, REQ-9)	Menú visible en menos de 3 s en gama baja y sin pedir datos
TC-HU-02	HU-02 (REQ-10)	Decisión correcta → sonido festivo, animación y mensaje del decálogo en la tarjeta de hábito
TC-HU-03	HU-03 (REQ-3, REQ-5)	100 % offline y tamaño de la app *release* aceptable en Android e iOS
TC-HU-04	HU-04 (REQ-2)	En la tableta de ADO no se puede salir de la app y el juego se reinicia tras la inactividad
TC-HU-05	HU-05 (REQ-6)	0 % de tráfico de red y ningún nombre, correo, GPS o identificador guardado

3.3.2 Prueba alfa (evaluación heurística)

El equipo (4 evaluadores) revisa la app en dispositivos reales con las 10 heurísticas de Nielsen, adaptadas a niños: visibilidad del estado (estrellas, puntos, luz del semáforo), lenguaje del niño (textos del copiloto), control y libertad (Repetir, Salir), consistencia entre iOS y Android, prevención de errores (el auto frena solo antes de un peatón), reconocimiento en vez de memoria (íconos en lugar de texto), estética simple y ayuda (consejos del tutorial). Cada problema se califica de 0 (no es problema) a 4 (catástrofe) y se unifican los resultados.

3.3.3 Prueba de regresión

Pendiente hasta tener el código completo. Al cerrar cada tarea se volverán a correr:

- Las pruebas automáticas de Godot ya escritas durante el desarrollo: 27 casos del puente con `FakeBridge`, pruebas de cada regla manejando el auto y el recorrido completo de los niveles 0, 1 y 2 con un conductor automático.
- Los casos de aceptación del Excel marcados como críticos, en Android y en iOS.

Una prueba que antes pasaba y ahora falla bloquea la entrega de la tarea.

3.3.4 Pruebas dinámicas (caja blanca)

Se ejecuta el código revisando sus caminos internos, con escenas de prueba que corren en Godot sin interfaz (`--headless`) y el `FakeBridge` en lugar de la app nativa (H6). Módulos principales y qué se cubre:

Módulo	Archivo	Caminos que se cubren
Puente (lado Godot)	`autoload/bridge_client.gd`	Cada rama de validación de `start_level` (JSON inválido, versión, nivel, modo, campos), nivel encima de otro, pausa durante la pausa de una falta y con la tarjeta abierta, kiosco
Reglas y estrellas	`scenes/level/rule_manager.gd`	Cálculo de estrellas (0, 1–2, 3+ faltas), falta grave con regreso al punto de control, conteo por regla
Piezas de reglas	`scenes/rules/*.gd`	Una falta por pasada, reinicio al regresar a un punto de control, puntos solo si se cumplió la regla
Copiloto	`autoload/feedback.gd`	Pausa de 2 s solo en faltas, `clear()` sin despausar un menú abierto
Datos de nivel	`resources/level_data.gd`	Nivel inexistente, nivel sin escena, búsqueda por escena (F6)
Puente nativo	`RutaBridgePlugin.kt` / clase Swift `@Godot`	Las 5 señales y `notify()` en el hilo correcto

Meta: todas las decisiones (`if` / `match`) de estos módulos se ejecutan en los dos sentidos al menos una vez.

3.3.5 Pruebas funcionales (caja negra)

Se prueba solo con entradas y salidas, sin ver el código, usando **particiones de equivalencia** y **valores límite**. Las necesarias:

- **Límite de velocidad:** pasarse 1.9 s (sin falta) contra 2.1 s (falta); ir justo en el límite.
- **Estrellas:** terminar con 0, 1, 2, 3 y 4 faltas (3, 2, 2, 1 y 1 estrellas).
- **Alto:** detenerse por completo, casi detenerse y no detenerse.
- **Semáforo:** cruzar en rojo, en amarillo con y sin acelerar, y en verde después de esperar.
- **Paso peatonal:** ceder el paso, entrar con el peatón esperando y con el peatón cruzando.
- **Ciclovía:** quedarse en el carril, cambiarse a mitad del tramo y llegar ya dentro.
- **Puente:** mensajes válidos e inválidos de cada tipo, y la secuencia completa de un nivel desde el menú nativo.

3.3.6 Prueba de usabilidad con usuarios

Pendiente hasta tener el código completo. Participantes: niñas y niños de 4 a 7 años (modo peque) y de 8 a 13 (modo grande), tutores y personal de ADO. Cada participante hace tareas

como "abre el juego y termina el nivel del parque" o "repite un nivel". Se mide: si termina la tarea, cuánto tarda, cuántas veces pide ayuda y qué tan claro le resultó el copiloto. Se hace con consentimiento de los tutores y sin grabar ni guardar datos personales de los menores.

3.3.7 Pruebas no funcionales

Además de la usabilidad:

1. **Seguridad y privacidad** (REQ-5, REQ-6, HU-05):
   - Capturar el tráfico de red mientras se juega: debe ser 0 bytes.
   - Revisar los permisos finales de la app (manifiesto de Android ya combinado con `godot-lib`, `Info.plist` en iOS): ninguno de red, ubicación, cámara ni contactos. Primer resultado (8 oct, APK *debug* del spike de Android): `aapt2 dump permissions` no muestra ningún permiso aparte del interno que agrega Android (`DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION`); no hay `INTERNET`.
   - Revisar lo que se guarda en el dispositivo: solo estrellas, puntos, faltas por regla y ajustes.
   - Kiosco (HU-04): intentar salir de la app en la tableta de ADO.
2. **Rendimiento** (REQ-4, HU-01, HU-03): tiempo hasta ver el menú (meta: menos de 3 s), tiempo hasta `engine_ready`, fps durante un nivel (meta: 60, nunca menos de 30), memoria en 10 ciclos de abrir y cerrar, y tamaño de la app *release*.
3. **Compatibilidad** (REQ-1): el mismo flujo en Android arm64 y armv7, y en iOS; mismos resultados del puente (paridad, H5).

3.4 Dispositivos

- **Android:** Samsung Galaxy A15 (SM-A155M, GPU Mali-G57). Ahí ya se corrió el spike: menú visible en 1.9 s, `engine_ready` en 3.7 s y memoria de 247 → 253 MB en 10 ciclos, todo en build *debug*.
- **iOS:** iPhone o iPad real, porque Godot no corre en el simulador (H3). Pendiente.

---

4. Exclusiones

No entra en este plan:

1. **El simulador de iOS y el emulador de Android como medio de validación.** En el simulador de iOS Godot no corre (H3). En el emulador de Android la GPU por software no compila los shaders 2D de Godot. Solo cuentan los dispositivos reales.
2. **El modo "peque" (el auto avanza solo):** todavía no está implementado.

3. **Niveles 3 a 10 y sus reglas** (cruce sin vista, paciencia, amarillo, celular, luces, distancia, ambulancia): aún no se construyen.
4. **La versión 100% Godot** (menús, SaveData y kiosco dentro de Godot). Solo se probaría si el spike falla y se descarta la arquitectura híbrida (H13).
5. **El contenido final de textos y voces:** las frases son borrador hasta que las valide Fundación ADO, y los audios de voz todavía no existen.
6. **El arte final:** se prueba con tiles provisionales y los assets de la fase 1.
7. **La generación automática de textos para iOS y Android** desde `textos.csv` (H8) y el script de exportación del `.pck` para iOS (H10): se probarán cuando existan.
8. **El bloqueo de pantalla del kiosco a nivel sistema operativo** (lock task en Android, Acceso Guiado en iPad): es configuración del dispositivo, no del juego.
9. **Pruebas de carga de red y de servidores:** la app no usa internet ni tiene servidores (REQ-5). La seguridad y privacidad del dispositivo sí se prueban (3.3.7).
10. **La publicación en tiendas** (App Store, Google Play) y la firma de producción.

Nombre de la prueba	Responsable	Calendario	Necesidades ambientales	Criterios de aceptación
1. Validación de requerimientos (un caso por REQ-1 a REQ-10 y HU-01 a HU-05)	Los 4, según el Excel: **Dylan Isaac** (REQ-1, 4, 6, 7, 9), **Paulo Vélez** (REQ-1, 3, 5, 6, 8, 10), **Diego Sánchez** (REQ-2, 6, 8, 10), **Ángel Mael** (REQ-4, 5, 7, 9). Aprobador: **Fundación ADO**	Diseño de casos: 07/10/26 – 09/10/26 (*Verificación de requerimientos*, SRS). Ejecución: 20/10/26 – 25/10/26 (Fase 3). Cierre: **25/10/26 (Hito 4)**	Build de pruebas en Android gama baja (Samsung Galaxy A15) e iPhone/iPad real; tableta Android e iPad para kiosco; Excel de 25 casos UAT	Los 25 casos UAT del Excel en estado "Aprobado" (al menos 2 por requerimiento) y cada HU cumple sus criterios de aceptación del SRS
2. Prueba alfa (evaluación heurística)	**Paulo Vélez**	16/10/26 – 18/10/26 (*Identificación de problemas*, SRS)	Build en Galaxy A15 e iPhone/iPad; formato con las 10 heurísticas de Nielsen adaptadas a	Ningún problema de severidad 3 o 4 sin corregir al 22/10/26 (fin de *Correcciones*)

Nombre de la prueba	Responsable	Calendario	Necesidades ambientales	Criterios de aceptación
			niños; escala de severidad 0–4	
3. Regresión *(pendiente hasta tener el código)*	**Ángel Mael**	Después de cada corrección: 19/10/26 – 22/10/26 (*Correcciones*); de nuevo en Fase 4 (26/10/26 – 08/11/26) al cambiar arte y exportar a iOS	Godot 4.7.2 en modo sin interfaz (`--headless`) con `FakeBridge` ; pruebas automáticas del repositorio; Galaxy A15 por Wi-Fi debugging; iPhone/iPad real	El 100 % de las pruebas que pasaban antes vuelve a pasar; una prueba que falla bloquea la entrega de la tarea
4. Pruebas dinámicas (caja blanca)	Lado Godot y Android: **Paulo Vélez** y **Ángel Mael** Puente iOS: **Diego Sánchez** y **Dylan Isaac**	10/10/26 – 15/10/26 (*Pruebas de caja negra/blanca, plataforma*, SRS)	Repositorio en GitHub; Godot 4.7.2 `--headless` ; escenas de prueba y `FakeBridge` ; Android Studio; Xcode en Mac	Todas las decisiones (`if` / `match`) de `bridge_client.gd` , `rule_manager.gd` , piezas de reglas, `feedback.gd` y `level_data.gd` se ejecutan en los dos sentidos; 0 pruebas fallidas
5. Pruebas funcionales (caja negra)	Coordina: **Paulo Vélez** Participa Diego y Angel	10/10/26 – 15/10/26 (*Pruebas de caja negra/blanca*, SRS)	Build en Galaxy A15; niveles 0, 1, 2 y nivel de prueba 99; cronómetro para los valores límite	Cada partición y valor límite da el resultado esperado: p. ej. 1.9 s arriba del límite sin falta y 2.1 s con falta; 0 / 1–2 / 3+ faltas → 3 / 2 / 1 estrellas
6. Usabilidad con usuarios	**Diego Sánchez**	23/10/26 – 25/10/26	Tutores con autorización	80 % o más de cada grupo termina el

Nombre de la prueba	Responsable	Calendario	Necesidades ambientales	Criterios de aceptación
(pendiente hasta tener el código)		(*Verificación UX*, SRS)	firmada; 3 grupos de edad (4–6, 7–9 y 10–13 años, 5 o más por grupo); Galaxy A15 y tableta; formato de observación	Nivel 1 sin ayuda; los controles se entienden sin explicación
7a. No funcional: seguridad y privacidad	**Dylan Isaac Diego Sánchez Paulo Vélez**	20/10/26 – 25/10/26 (Fase 3)	PCAPdroid en Android y Charles Proxy en iOS; modo avión; `aapt2 dump permissions`; acceso a archivos de la app; tableta en modo kiosco	0 peticiones a servidores y 0 consultas DNS; ningún permiso de red, ubicación, cámara o contactos; solo estrellas, puntos, faltas por regla y ajustes guardados; no se puede salir del kiosco sin el método del personal
7b. No funcional: rendimiento	**Dylan Isaac Ángel Mael Paulo Vélez**	20/10/26 – 25/10/26 (Fase 3); el SRS pide terminar las de rendimiento **antes del 25/10/26**	Galaxy A15 (gama baja); medidor de fps; cronómetro; build *release* (requiere el NDK de Android para el tamaño real); bitácora de sesiones	Menú visible en **3 s o menos** (10 aperturas en frío); **30 fps o más**; **menos de 2 % de sesiones con cierre** (50 sesiones, 0 cierres); tamaño de **150 MB o menos** (propuesto en el Excel)

Nombre de la prueba	Responsable	Calendario	Necesidades ambientales	Criterios de aceptación
7c. No funcional: compatibilidad	**Ángel Mael Dylan Isaac Paulo Vélez**	Android: 10/10/26 – 15/10/26 (*Plataforma*, SRS). iOS: 26/10/26 – 08/11/26 (Fase 4, exportar desde la Mac)	Matriz de 10 o más dispositivos acordada con ADO (8 o más Android de distintas marcas y versiones, más iPhone e iPad); builds arm64 y armv7	El 90 % o más de la matriz pasa el recorrido básico (9 de 10); el puente da los mismos resultados en iOS y Android (paridad, H5)